Abstract Diagrams for the Working Mathematician

Roman Maksimovich

Abstract

Within the niche of mathematicians, there's a niche of those who need to draw diagrams in pure mathematics. Within *that* niche there's a niche of people who have experience in programming and feel comfortable writing code to produce graphics. This article is for them. It's a survey of the available tools and a showcase of a library I wrote specifically to simplify the process of drawing diagrams in differential geometry, topology, set theory, etc.

1 Introduction

Back in high school, I was attending a seminar on differential geometry, and I wanted to practice my LaTeX skills by typesetting the lecture notes. I found myself needing to draw simple surfaces such as those seen on Figure 1.

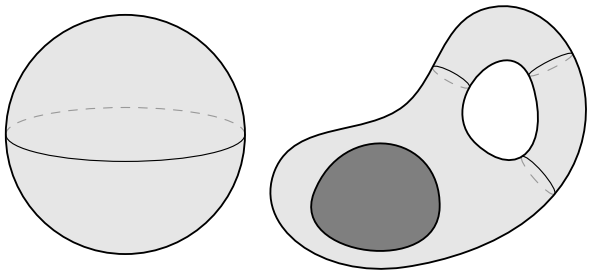

Figure 1: Basic two-dimensional manifolds

At that point I realized I wasn't sure how to draw this. I started by considering some of the available tools:

- **Inkscape** (`https://www.inkscape.org`)
  This is a popular and versatile GUI program for drawing vectorized graphics, suitable when the diagram is simple enough to be drawn by hand with some precision assistance. This assumption holds for the left-hand side of Figure 1, but not for the right-hand side. The reason is that each "cross section ring" has to be drawn at a precisely calculated position, where it meets its bounding paths at equal angles, as illustrated on Figure 2.
  Besides, it would be useful to tweak the count and positions of these "cross sections" to see what works best for the picture, and this is not something that can be achieved with a GUI tool like **Inkscape**. Pass.
- **LaTeX's `picture` environment**
  There are native drawing capabilities to LaTeX,

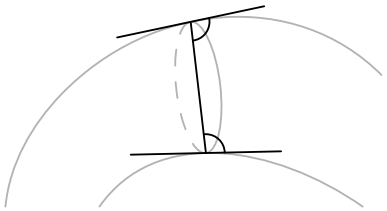

Figure 2: A good position for drawing a cross section ring, equal angles with the tangent lines

but they are severely limited. For instance, a straight line segment can be drawn like so:

```
\begin{picture}(15,30)
  \put(0,0){\line(1,2){15}}
\end{picture}
```

This yields the line shown on the right-hand side. The command `\line(m,n){l}` creates a line whose slope is `n`/`m` and whose horizontal projection is `l` units long. However, both `n` and `m` must not exceed 6, and must be coprime. The reason for these weird restrictions is the mechanism behind the `picture` environment: instead of producing true vector graphics, it tries to assemble the diagram from glyphs found in TeX's fonts, and these fonts only provide a small variety of sloped straight lines (if you are reading this article digitally, you can zoom in on the straight line above and see that it's actually three aligned forward slashes).
To be fair, `picture` is also capable of drawing an arbitrary Bézier curve by putting it together from many black squares. Still, this is a pretty rudimentary tool unsuited for complicated technical drawing, and it's mainly used to define custom symbols rather than full diagrams. Pass.
- **The Ti*k*Z package** (`https://tikz.dev`)
  For the absolute majority of TeX users, this is the go-to tool for technical drawing, and reasonably so. Ti*k*Z has a comprehensive manual and a rich ecosystem of packages to suit virtually your every need. Clearly, both manifolds on Figure 1 can be drawn with Ti*k*Z.
  However, my high school self was positively intimidated by the Ti*k*Z manual (1321 pages?!), and besides, I wanted to write an *algorithm* that could calculate the best positions for cross section rings, not just a TeX macro. I was fascinated by the **C** programming language at the time, and I wanted to write a *function* which would accept two paths as arguments, as well as the desired number of cross sections, and return the coordinates at which they would look best. Ti*k*Z certainly tries to provide the features of

a full programming language, but in truth it is more of a markup language, poorly suited for writing general-purpose algorithms. Hence, I passed on Ti*k*Z also. Alternatives to the Ti*k*Z package (such as `pstricks`, `xypic`, `dratex`, etc.) suffer from the same and more disadvantages.

Then, in a stroke of luck, I heard about this vector graphics program called **Asymptote** (`https://asymptote.sourceforge.io`), and I knew that it was the one. **Asymptote** was initially introduced in 2004 at the University of Alberta by Andy Hammerlindl, John Bowman, and Tom Prince. It is based on the old **MetaPost** drawing program, which in turn was based on Donald Knuth's **MetaFont**. However, **Asymptote** has a **C**-like syntax and is indeed a full programming language with data structures, IEEE floating-point number support, control flow structures like `if`, `for`, and `while`, etc., as well as very high-level features such as default arguments and first-class functions (i.e. functions that can be returned or passed to other functions as values).
Soon after learning about **Asymptote**, I started writing my own library for this language, one that I later came to use whenever I needed to create a diagram in abstract mathematics.

2 Module `smoothmanifold`

2.1 The basics of Asymptote

An **Asymptote** user produces graphics by writing code in a file with the `.asy` extension, then invoking the `asy` program on the command line to compile the code to a picture in a format such as PDF, SVG, PostScript, or a number of others. Additionally, one may write **Asymptote** code directly in a LaTeX document by means of the `asymptote` package. This approach removes the necessity to manually include images, but the code becomes harder to debug.

An example code snippet can look like this:

```
settings.outformat = "eps";
size(4cm);
path p = (0,1) .. (1,0) .. (2,1);
path q = (0,0) .. (1,1) .. (2,0);
draw(p ^^ q);
pair[] points = intersectionpoints(p, q);
for (int i = 0; i < points.length; ++i) {
  dot(L = Label((string)i, align = S),
    z = points[i], linewidth(4pt));
}
```

The result of compiling this code is an Encapsulated PostScript file shown in Figure 3. Take note of **Asymptote**'s support of arrays and its native `intersectionpoints` function (whereas in Ti*k*Z one would load additional libraries to gain this functionality).

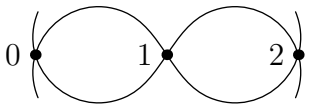

Figure 3: A sample picture produced by **Asymptote**

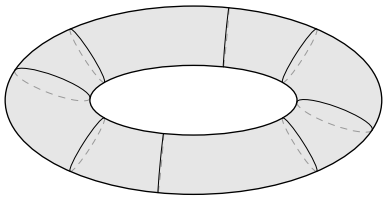

Figure 4: A torus

Asymptote has other outstanding features such as powerful three-dimensional drawing, rich support for mathematical functions, but most importantly, the ease with which the language can be extended or modified. In the following subsections, I will outline the most prominent features of my own library (which now counts more than 6000 lines of code) and how they allow for effortless drawing of abstract diagrams.

2.2 Smooth objects and cross sections

As you recall, my original goal was drawing two-dimensional manifolds like those in Figure 1, so this was the first functionality I programmed into my library, `smoothmanifold`. A simple torus can be drawn by writing

```
import smoothmanifold;
config.section.freedom = .9;
size(5cm);
smooth s = smooth(
  ellipse((0,0), 2, 1)
).addhole(
  ellipse((0,0), 1.1, .37),
  sections = new real[][]{
    r(0, 60, 3), r(90, 90, 1),
    r(180, 60, 3), r(270, 90, 1)
  }
);
draw(s, dpar(mode=free, viewdir=dir(90)));
```

Compiling this code (assuming that the file `smoothmanifold.asy` is located where **Asymptote** can find it) yields the picture in Figure 4.

◇ Roman Maksimovich
Hong Kong
`r.a.maksimovich (at) gmail.com`